

# OPERATING SYSTEM CONCEPTS LAB EXPERIMENTS

## EXP - 04: SYSTEM CALLS

```

Activities Terminal Mar 31 11:31
user123@localhost:~/system_calls

[user123@localhost ~]$ ls
abc.sh Desktop Downloads ifelse.sh Music Pictures Public REC sample.sh scheduling sh Templates while.sh
ab.sh Documents Folder loops.sh pc posneg.sh qns.txt sample.c sample.txt SFA system_calls Videos
[user123@localhost ~]$ cd system_calls
[user123@localhost system_calls]$ vi pid.c

```

```

Activities Terminal Mar 31 11:32
user123@localhost:~/system_calls — /usr/bin/vim pid.c

#include <stdio.h>
#include <unistd.h>

int main(){
    printf("PID: %d\n", getpid());
    printf("PPID: %d\n", getppid());

    return 0;
}

```

```

Activities Terminal Mar 31 11:34
user123@localhost:~/system_calls

[user123@localhost ~]$ ls
abc.sh Desktop Downloads ifelse.sh Music Pictures Public REC sample.sh scheduling sh Templates while.sh
ab.sh Documents Folder loops.sh pc posneg.sh qns.txt sample.c sample.txt SFA system_calls Videos
[user123@localhost ~]$ cd system_calls
[user123@localhost system_calls]$ vi pid.c
[user123@localhost system_calls]$ gcc pid.c
[user123@localhost system_calls]$ ./a.out
PID: 2802
PPID: 2505
[user123@localhost system_calls]$ vi fork.c

```

```

Activities Terminal Mar 31 11:35
user123@localhost:~/system_calls — /usr/bin/vim fork.c

#include <stdio.h>
#include <unistd.h>

int main(){
    if(fork()==0){
        printf("This is CHILD process\n");
    } else {
        printf("This is PARENT process\n");
    }

    printf("PID: %d\n", getpid());

    return 0;
}

```

```

Activities Terminal Mar 31 11:37
user123@localhost:~/system_calls

[user123@localhost ~]$ ls
abc.sh Desktop Downloads ifelse.sh Music Pictures Public REC sample.sh scheduling sh Templates while.sh
ab.sh Documents Folder loops.sh pc posneg.sh qns.txt sample.c sample.txt SFA system_calls Videos
[user123@localhost ~]$ cd system_calls
[user123@localhost system_calls]$ vi pid.c
[user123@localhost system_calls]$ gcc pid.c
[user123@localhost system_calls]$ ./a.out
PID: 2802
PPID: 2505
[user123@localhost system_calls]$ vi fork.c
[user123@localhost system_calls]$ gcc fork.c
[user123@localhost system_calls]$ ./a.out
This is PARENT process
PID: 2824
This is CHILD process
PID: 2825
[user123@localhost system_calls]$ vi exec.c

```

```
Activities Terminal Mar 31 11:39
user123@localhost:~/system_calls — /usr/bin/vim exec.c

#include <stdio.h>
#include <unistd.h>

int main(){
    printf("Before exec()\n");
    execl("/bin/ls", "ls", NULL);
    printf("After exec()\n");

    return 0;
}
```

```
Activities Terminal Mar 31 11:41
user123@localhost:~/system_calls

[user123@localhost ~]$ ls
abc.sh Desktop Downloads ifelse.sh Music Pictures Public REC sample.sh scheduling sh Templates while.sh
ab.sh Documents Folder loops.sh pc posneg.sh qns.txt sample.c sample.txt SFA system_calls Videos
[user123@localhost ~]$ cd system_calls
[user123@localhost system_calls]$ vi pid.c
[user123@localhost system_calls]$ gcc pid.c
[user123@localhost system_calls]$ ./a.out
PID: 2802
PPID: 2505
[user123@localhost system_calls]$ vi fork.c
[user123@localhost system_calls]$ gcc fork.c
[user123@localhost system_calls]$ ./a.out
This is PARENT process
PID: 2824
This is CHILD process
PID: 2825
[user123@localhost system_calls]$ vi exec.c
[user123@localhost system_calls]$ gcc exec.c
[user123@localhost system_calls]$ ./a.out
Before exec()
a.out exec.c fork.c opendir.c pid.c readdir.c
[user123@localhost system_calls]$ vi opendir.c
```

```
Activities Terminal Mar 31 11:41
user123@localhost:~/system_calls — /usr/bin/vim opendir.c

#include <stdio.h>
#include <dirent.h>

int main(){
    DIR *dir;

    dir = opendir(".");

    if(dir == NULL){
        printf("Failed to open directory\n");
        return 1;
    }

    printf("Directory opened successfully!\n");
    closedir(dir);
    return 0;
}
```

```
Activities Terminal Mar 31 11:42
user123@localhost:~/system_calls

[user123@localhost ~]$ ls
abc.sh Desktop Downloads ifelse.sh Music Pictures Public REC sample.sh scheduling sh Templates while.sh
ab.sh Documents Folder loops.sh pc posneg.sh qns.txt sample.c sample.txt SFA system_calls Videos
[user123@localhost ~]$ cd system_calls
[user123@localhost system_calls]$ vi pid.c
[user123@localhost system_calls]$ gcc pid.c
[user123@localhost system_calls]$ ./a.out
PID: 2802
PPID: 2505
[user123@localhost system_calls]$ vi fork.c
[user123@localhost system_calls]$ gcc fork.c
[user123@localhost system_calls]$ ./a.out
This is PARENT process
PID: 2824
This is CHILD process
PID: 2825
[user123@localhost system_calls]$ vi exec.c
[user123@localhost system_calls]$ gcc exec.c
[user123@localhost system_calls]$ ./a.out
Before exec()
a.out exec.c fork.c opendir.c pid.c readdir.c
[user123@localhost system_calls]$ vi opendir.c
[user123@localhost system_calls]$ gcc opendir.c
[user123@localhost system_calls]$ ./a.out
Directory opened successfully!
[user123@localhost system_calls]$ vi readdir.c
```

```
Activities Terminal Mar 31 11:44
user123@localhost:~/system_calls — /usr/bin/vim readdir.c

#include <stdio.h>
#include <dirent.h>

int main(){
    DIR *dir;
    struct dirent *entry;

    dir = opendir(".");

    if(dir == NULL){
        printf("Cannot open directory\n");
        return 1;
    }

    printf("Files in directory:\n");

    while((entry=readdir(dir)) != NULL){
        printf("%s\n", entry->d_name);
    }

    closedir(dir);

    return 0;
}
~
~
```

```
Activities Terminal Mar 31 11:45
user123@localhost:~/system_calls

[user123@localhost ~]$ ls
abc.sh Desktop Downloads ifelse.sh Music Pictures Public REC sample.sh scheduling sh Templates while.sh
ab.sh Documents Folder loops.sh pc posneg.sh qns.txt sample.c sample.txt SFA system_calls Videos
[user123@localhost ~]$ cd system_calls
[user123@localhost system_calls]$ vi pid.c
[user123@localhost system_calls]$ gcc pid.c
[user123@localhost system_calls]$ ./a.out
PID: 2802
PPID: 2505
[user123@localhost system_calls]$ vi fork.c
[user123@localhost system_calls]$ gcc fork.c
[user123@localhost system_calls]$ ./a.out
This is PARENT process
PID: 2824
This is CHILD process
PID: 2825
[user123@localhost system_calls]$ vi exec.c
[user123@localhost system_calls]$ gcc exec.c
[user123@localhost system_calls]$ ./a.out
Before exec()
a.out exec.c fork.c opendir.c pid.c readdir.c
[user123@localhost system_calls]$ vi opendir.c
[user123@localhost system_calls]$ gcc opendir.c
[user123@localhost system_calls]$ ./a.out
Directory opened successfully!
[user123@localhost system_calls]$ vi readdir.c
[user123@localhost system_calls]$ gcc readdir.c
[user123@localhost system_calls]$ ./a.out
Files in directory:
.
..
pid.c
fork.c
exec.c
opendir.c
readdir.c
a.out
[user123@localhost system_calls]$
```